



SIMATS ENGINEERING
CO5-ASSESSMENT TOOL -1
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



Student Name: TALARI VISHNUVARDHAN

Reg. No.: 192572091

Course Code: CSA0613

Slot: A

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

Course Faculty: Dr. Mary Harin Fernandez F

1. Latin Square Completion Using Constraint Satisfaction and Backtracking

2. PROBLEM STATEMENT

A **Latin Square** is an $n \times n$ grid filled with n different symbols such that every symbol appears **exactly once in each row and exactly once in each column**.

In this problem, some cells of the Latin Square are already filled, while others are empty. The objective is to complete the square while satisfying all row and column uniqueness constraints.

The problem is modeled as a **Constraint Satisfaction Problem (CSP)** and solved using a **backtracking-based search algorithm with constraint checking**.

3. INDUSTRIAL PROBLEM SCENARIO

Constraint Satisfaction Problems are widely used in industries where multiple decisions must satisfy a set of rules.

For example, consider a company that needs to assign:

- Employees to different shifts
- Machines to production tasks
- Students to examination rooms
- Resources to different projects

Each assignment must satisfy certain constraints.

The Latin Square Completion problem represents this type of situation because every value must be assigned without violating row and column constraints.

The same backtracking and constraint satisfaction techniques can be applied to:

- Employee scheduling
- Timetable generation
- Resource allocation

- Production scheduling
- Network frequency assignment

4. OBJECTIVES

The objectives of this project are:

1. To understand the Latin Square Completion problem.
2. To model the problem as a Constraint Satisfaction Problem.
3. To define row and column uniqueness constraints.
4. To implement a backtracking algorithm.
5. To apply constraint checking before assigning values.
6. To reduce the search space by rejecting invalid assignments.
7. To analyze the correctness of the solution.
8. To evaluate computational complexity.
9. To study real-world applications of constraint-based problem solving

5. INPUT LATIN SQUARE

The following partially filled 4×4 Latin Square is used:

1 0 3 4

0 4 1 2

2 1 0 3

4 3 2 0

Here:

0 = Empty Cell

The program must replace all 0 values with valid numbers.

6. CONSTRAINTS

The Latin Square follows the following constraints.

Constraint 1: Row Uniqueness

Every number must appear exactly once in each row.

Example:

1 2 3 4

All values are unique.

Constraint 2: Column Uniqueness

Every number must appear exactly once in each column.

Example:

1

3

2

4

No value is repeated

Constraint 3: Valid Symbols

For a 4×4 Latin Square, only the following values are allowed:

1, 2, 3, 4

7. CSP REPRESENTATION

The problem can be represented as a Constraint Satisfaction Problem.

Variables

The empty cells in the square.

Domain

Each empty cell can contain:

$\{1, 2, 3, 4\}$

Constraints

Each value must:

- Not repeat in the same row.
- Not repeat in the same column.

8. ALGORITHM USED

Backtracking with Constraint Checking

Backtracking is used to systematically search for a valid solution.

The algorithm:

1. Finds an empty cell.
2. Tries possible values.

3. Checks row and column constraints.
4. Assigns a valid value.
5. Moves to the next empty cell.
6. If no valid value is available, it returns to the previous cell.
7. Changes the previous assignment.

This process continues until a valid Latin Square is found.

9. ALGORITHM / PSEUDOCODE

START

Find an empty cell

IF no empty cell exists

 Solution found

 STOP

FOR each number from 1 to n

 Check whether the number is valid

 IF valid

 Place the number

 Recursively solve remaining cells

 IF solution found

 Return True

 Remove the number

Return False

STOP

10. PYTHON SOURCE CODE

```
1 square = [  
2     [1, 0, 3, 4],  
3     [0, 4, 1, 2],  
4     [2, 1, 0, 3],  
5     [4, 3, 2, 0]  
6 ]  
7 def valid(row, col, num):  
8     if num in square[row]:  
9         return False  
10    for i in range(4):  
11        if square[i][col] == num:  
12            return False  
13    return True  
14 def solve():  
15    for row in range(4):  
16        for col in range(4):  
17            if square[row][col] == 0:  
18                for num in range(1, 5):  
19                    if valid(row, col, num):  
20                        square[row][col] = num  
21                        if solve():  
22                            return True  
23                        square[row][col] = 0  
24                return False  
25    return True  
26 solve()  
27 print("Completed Latin Square:")  
28 for row in square:  
29     print(row)
```

11. EXPECTED OUTPUT

```
Completed Latin Square:  
[1, 2, 3, 4]  
[3, 4, 1, 2]  
[2, 1, 4, 3]  
[4, 3, 2, 1]
```

12. WORKING OF THE PROGRAM

Consider the first row:

1 0 3 4

The missing number is:

2

Therefore:

1 2 3 4

Now consider the second row:

0 4 1 2

The missing number is:

3

Therefore:

3 4 1 2

The algorithm continues this process until all empty cells are filled.

13. CONSTRAINT PROPAGATION

Constraint propagation reduces the number of possible choices.

For example:

1 0 3 4

The possible domain is:

{1, 2, 3, 4}

But:

- 1 already exists in the row.
- 3 already exists in the row.
- 4 already exists in the row.

Therefore, only:

2

is possible.

This reduces the search space.

14. BACKTRACKING PROCESS

Suppose the algorithm tries a value.

Step 1

Assign a possible value.

1 2 3 4

Step 2

Check the constraints.

If all constraints are satisfied:

Continue to the next cell.

If constraints are violated:

Remove the value.

Step 3

Try another value.

This process is called:

Backtracking

15. SEARCH SPACE REDUCTION

Without constraints, every empty cell could have multiple possible values.

For example:

Domain = {1, 2, 3, 4}

However, row and column constraints eliminate invalid values.

Therefore:

Initial Domain

↓

Apply Row Constraint

↓

Apply Column Constraint

↓

Valid Values Only

This significantly reduces unnecessary searching.

16. CORRECTNESS OF THE ALGORITHM

The algorithm is correct because it checks every assignment before placing it.

A value is placed only if:

Condition 1

The value does not already exist in the row.

Condition 2

The value does not already exist in the column.

Therefore, every completed row satisfies:

All values are unique

And every completed column satisfies:

All values are unique

If an assignment leads to an invalid situation, the algorithm removes it and tries another value.

Thus, if the algorithm returns a solution, it is guaranteed to be a valid Latin Square.

17. COMPUTATIONAL COMPLEXITY

Let:

n = Size of Latin Square

The algorithm may try several possible values for every empty cell.

Worst Case Time Complexity

The worst-case search can grow exponentially.

$O(n^{(n^2)})$

However, practical performance is improved using:

- Constraint checking
- Backtracking
- Search space reduction

Space Complexity

The square requires:

$O(n^2)$

The recursive calls require additional memory.

Therefore, the overall space requirement depends on the square size and recursion depth.

18. ADVANTAGES OF CONSTRAINT SATISFACTION

The advantages include:

1. Flexible

New constraints can easily be added.

2. Accurate

Only valid assignments are accepted.

3. Efficient Search

Invalid solutions are rejected early.

4. General Purpose

The same approach can solve many industrial problems.

5. Automatic Decision Making

The system can automatically find valid assignments.

19. INDUSTRIAL APPLICATIONS

1. Employee Scheduling

Employees can be assigned to shifts while ensuring:

- No employee receives conflicting shifts.
- Required employees are available.

2. Examination Timetabling

Exams must be scheduled without conflicts.

Constraints include:

- A student cannot write two exams simultaneously.
- Rooms must have sufficient capacity.

3. Resource Allocation

Resources are assigned to tasks while avoiding conflicts.

4. Production Scheduling

Machines are assigned to production jobs.

Constraints may include:

- Machine availability
- Production time
- Resource limitations

5. Frequency Assignment

Communication frequencies must be assigned without interference.

6. Sudoku Solving

Sudoku is also a Constraint Satisfaction Problem.

The solver must satisfy:

- Row constraints
- Column constraints

20. RESULT ANALYSIS

The program successfully completed the partially filled Latin Square.

Input

1 0 3 4

0 4 1 2

2 1 0 3

4 3 2 0

Output

1 2 3 4

3 4 1 2

2 1 4 3

4 3 2 1

The completed square satisfies all required constraints.

Verification

Every row contains numbers 1, 2, 3, 4 exactly once.

Every column contains numbers 1, 2, 3, 4 exactly once.

Therefore, the generated solution is a valid Latin Square.

21. CHALLENGES

Some challenges include:

Large Problem Size

As the size of the square increases, the search space increases rapidly.

Multiple Constraints

Additional constraints make the problem more complex.

Multiple Solutions

Some Latin Square problems may have more than one valid solution.

No Solution

Certain partially filled configurations may not have any valid solution.

22. POSSIBLE IMPROVEMENTS

The system can be improved using:

- Minimum Remaining Value (MRV)
- Forward Checking
- Advanced Constraint Propagation
- Heuristic Search
- Artificial Intelligence techniques

These methods can reduce the search space and improve performance.

23. COMPARISON OF METHODS

Method	Advantage	Limitation
Brute Force	Simple	Very slow
Backtracking	Reduces invalid search	Can be slow for large problems
Constraint Propagation	Reduces search space	More complex
Heuristic CSP	Faster for large problems	Requires better implementation

24. CONCLUSION

The **Latin Square Completion problem** was successfully modeled and solved as a **Constraint Satisfaction Problem using Backtracking**.

The solution applies row and column uniqueness constraints to ensure that every symbol appears exactly once in each row and column. Constraint checking eliminates invalid choices, while backtracking explores alternative assignments when necessary.

The project demonstrates an important concept in Artificial Intelligence and constraint-based problem solving. The same techniques can be applied to industrial problems such as employee scheduling, examination timetabling, resource allocation, production planning, and frequency assignment.

Thus, Constraint Satisfaction and Backtracking provide a powerful approach for solving complex optimization and decision-making problems.